

SHIV NADAR UNIVERSITY CHENNAI

School of Engineering

CS3807 – Deep Learning Laboratory

Experiment – 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Name : Srivatsan S
Register No. : 24011101093
Degree & Branch : B.Tech Artificial Intelligence & Data Science
Class / Section : AIDS – B
Semester : V
Subject Code : CS3807
Subject Name : Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

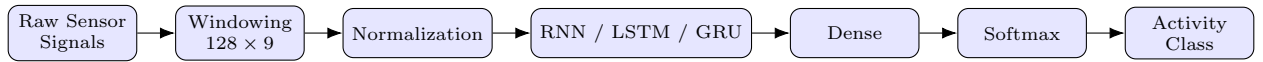
The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

Training : (N_train, 128, 9)

Validation : (N_val, 128, 9)

Testing : (N_test, 128, 9)

Number of classes: 6

Number of features per time step: 9

Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Sensor Signals Across Activity Classes

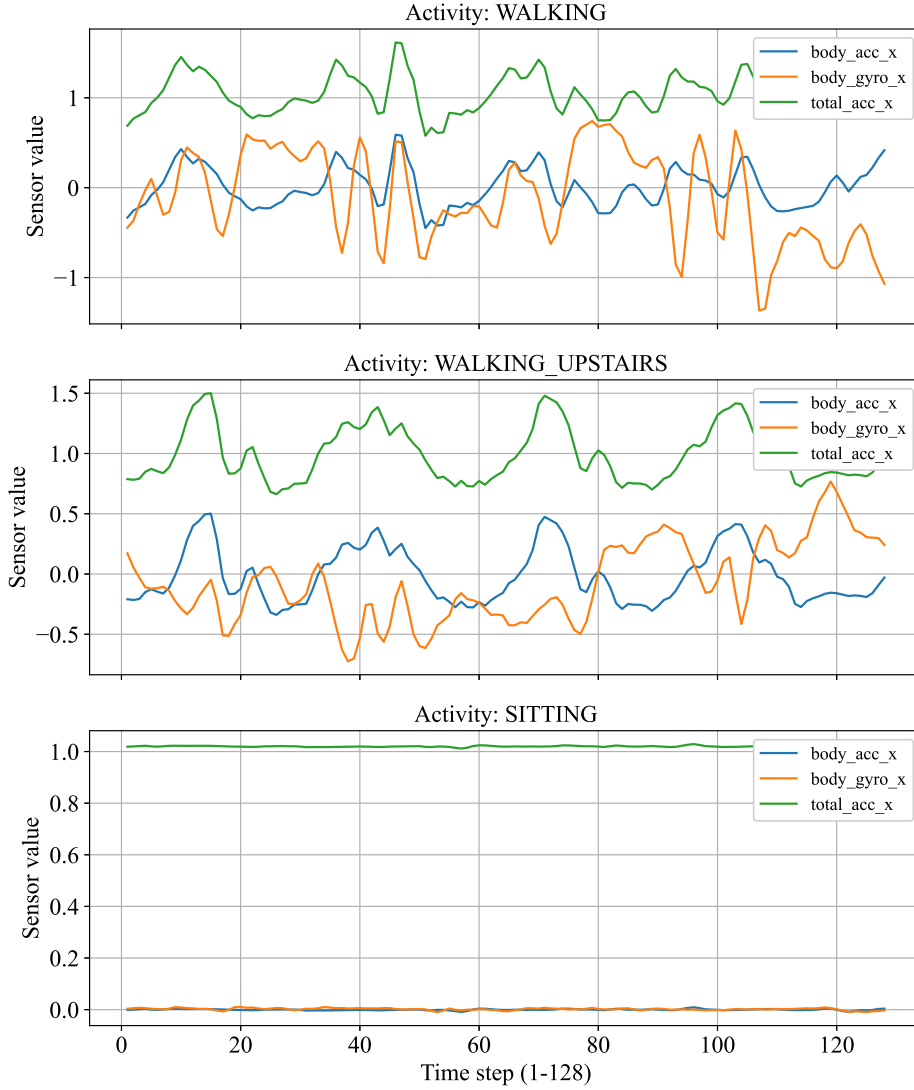


Figure 1: Temporal sensor signals by activity.

Inference: Stationary actions like sitting and standing have stable temporal variations whereas activities like walking and any movement have periodic fluctuations

7. Vanilla RNN Architecture

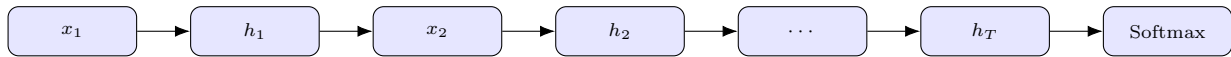
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

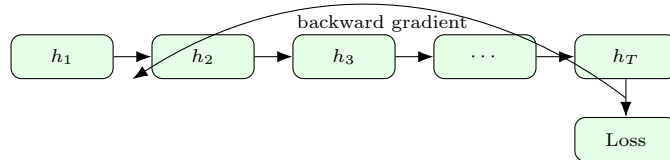
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

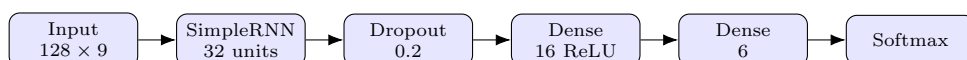
$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

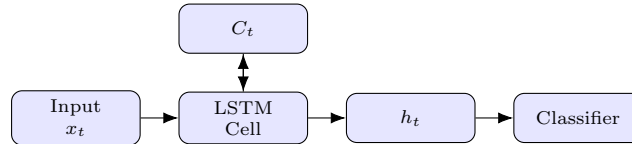
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

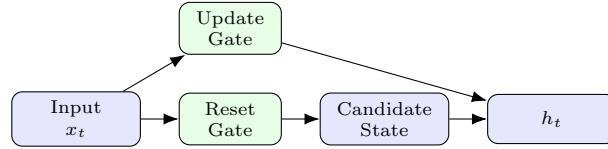
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t \tilde{h}_t.$$



12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

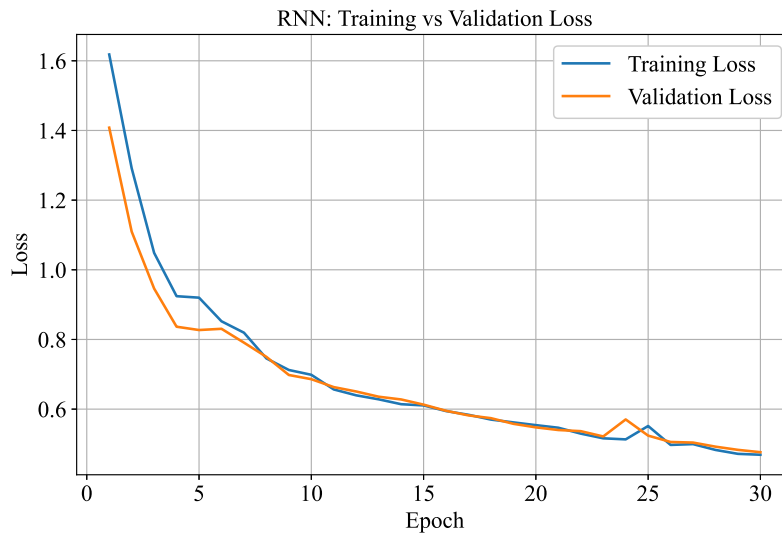


Figure 2: RNN training and validation loss.

Inference: The training and validation loss converges steadily from 1.6, 1.4 resp to a 0.5 over the course of the 30 epochs . Curves are closer hence no overfitting observed

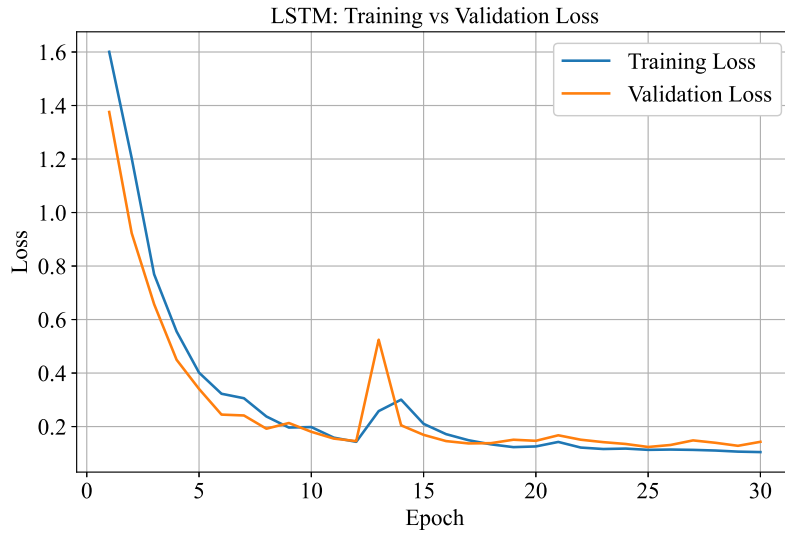


Figure 3: LSTM training and validation loss.

Inference: The training and validation loss converges steadily from 1.6,1.4 resp to a 0.2 over the course of the 30 epochs . Curves are closer hence no overfitting observed . This better performance is due to the lstm gates and cell states that enables effective learning of temporal dependencies

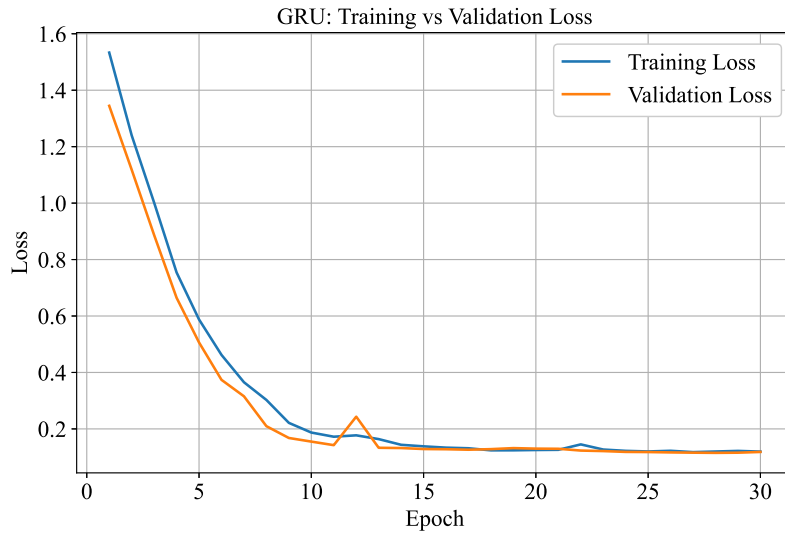


Figure 4: GRU training and validation loss.

Inference: The training and validation loss converges steadily from 1.6,1.4 resp to a 0.2 over the course of the 30 epochs . Curves are closer hence no overfitting observed . This converged faster than lstm with fewer parameters because the update and reset gates makes the gru retain and discard temporal information .

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

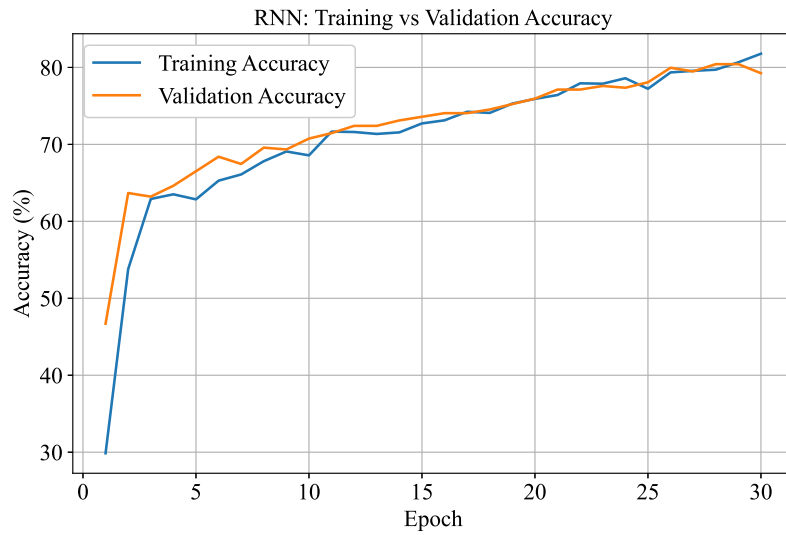


Figure 5: RNN training and validation accuracy.

Inference: The RNN reaches an accuracy of 80 which is lower and a slower convergence when compared to the lstm and the gru training . Still the training and validation curves are similar hence no overfitting

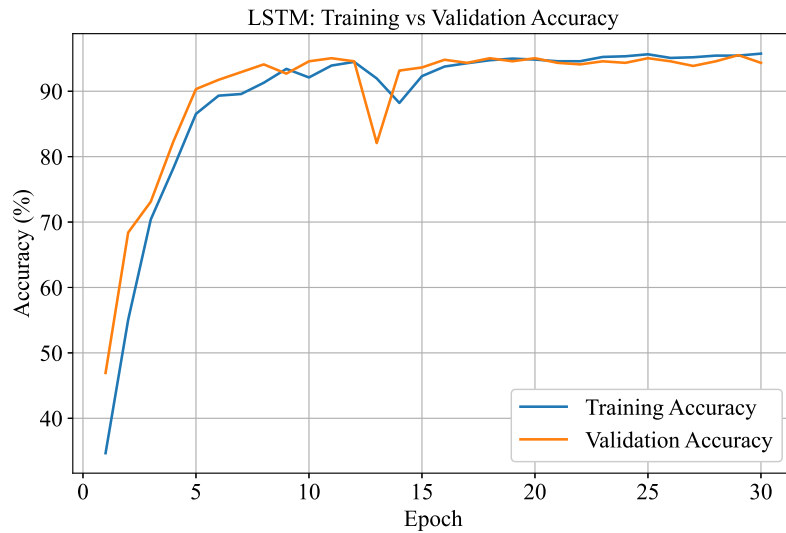


Figure 6: LSTM training and validation accuracy.

Inference: Lstm accuracy reaches faster to a 90 percent at around 6-7 epochs and remains almost same throughout , and the curves are largely similar hence it generalises better

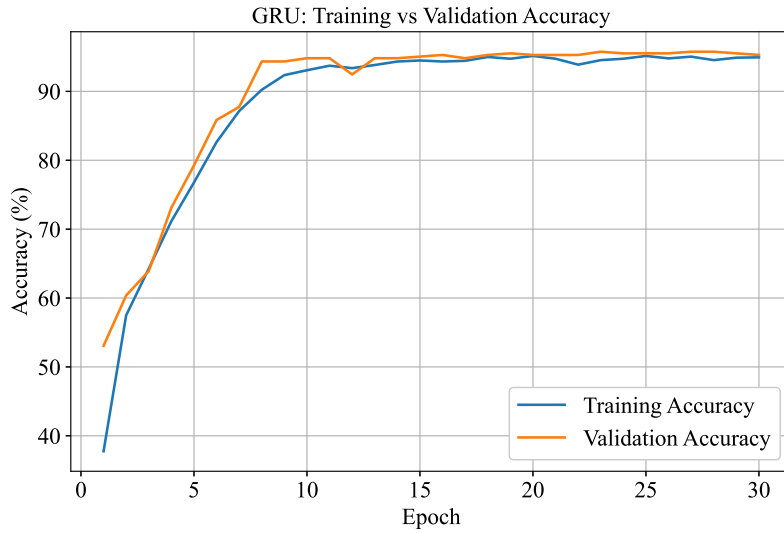


Figure 7: GRU training and validation accuracy.

Inference: Similar to the lstm the GRU accuracy raises to 90 at around 6-7 epochs and remains almost same with a good generalisation

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	73.057	89.277	90.431
Macro Precision (%)	73.107	89.275	90.431
Macro Recall (%)	72.108	89.375	90.550
Macro F1 (%)	72.316	89.208	90.414
Parameters	1974	6006	4758
Training Time (s)	33.45	25.67	24.45

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

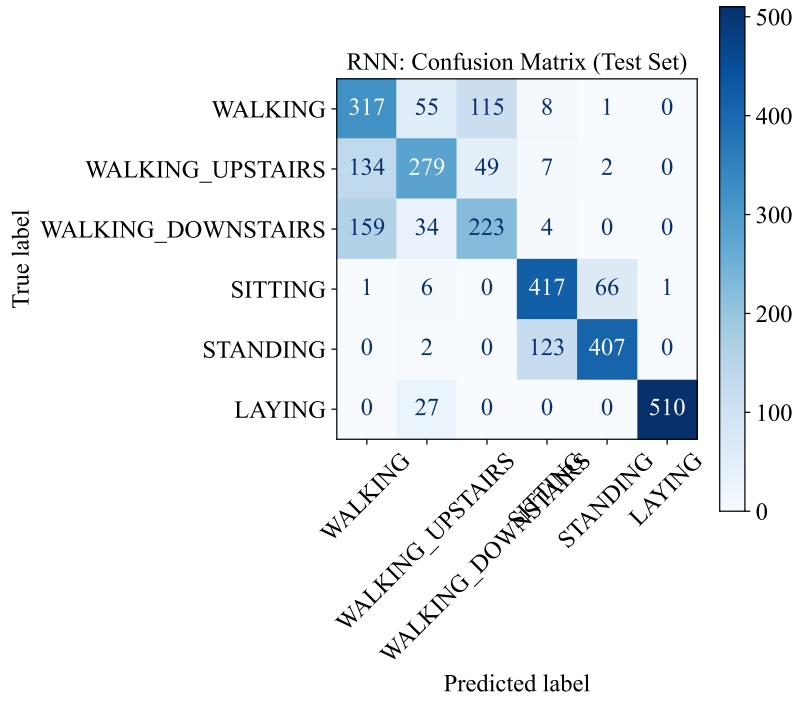


Figure 8: RNN confusion matrix.

Inference: We can see classes like walking , walking upstairs and walking downstairs have highly misclassified sample and even sitting and standing too having confusion this is because of the similar motion patterns in the camera across the actions.

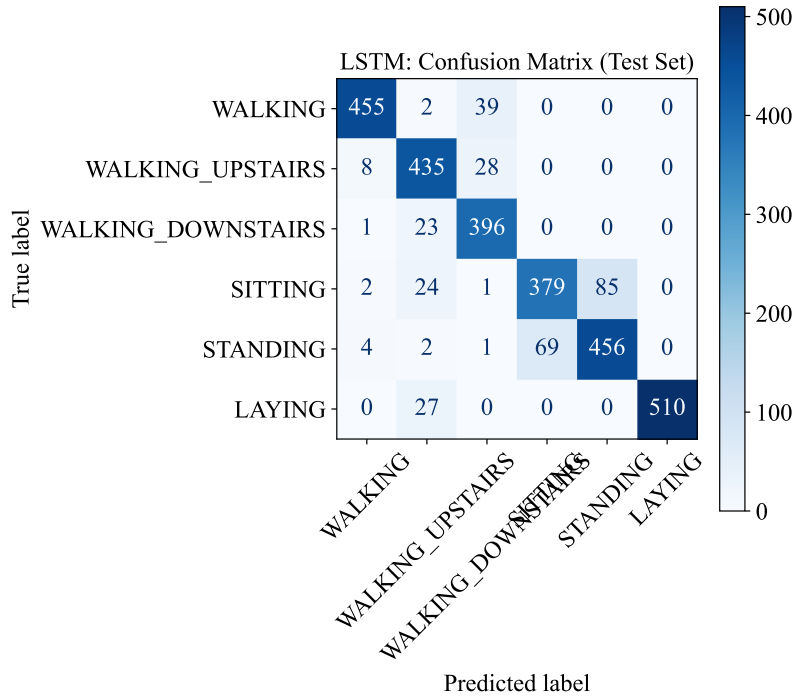


Figure 9: LSTM confusion matrix.

Inference: The LSTM has a relatively strong diagonal values signifying better true samples with somewhat confusion still remaining between sitting and standing

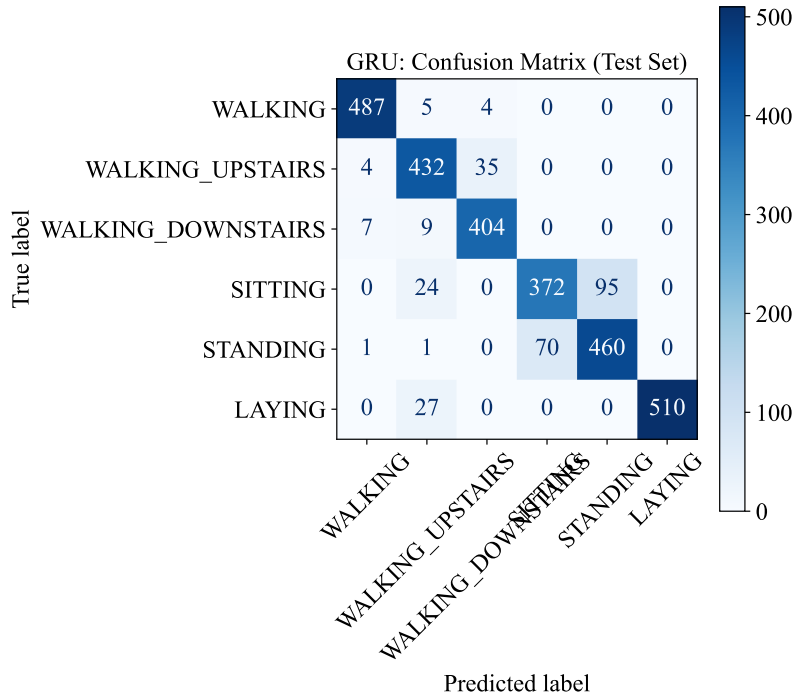


Figure 10: GRU confusion matrix.

Inference: The GRU similar to the lstm has very strong diagonal values when compared to a rnn but still faces the same problems of confusing between sitting and standing

- Laying has the highest recognition across all three models while walking downstairs, sitting and sitting were the highly misclassified classes for the rnn, lstm and gru respectively . For the rnn the most confused classes were walking downstairs, walking and walking upstairs whereas for the lstm and gru the confusion was between sitting and standing

16. RNN vs. LSTM vs. GRU Comparison

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time	33.45 s	25.67 s	24.45 s
Test F1-score	72.32%	89.21%	90.41%

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

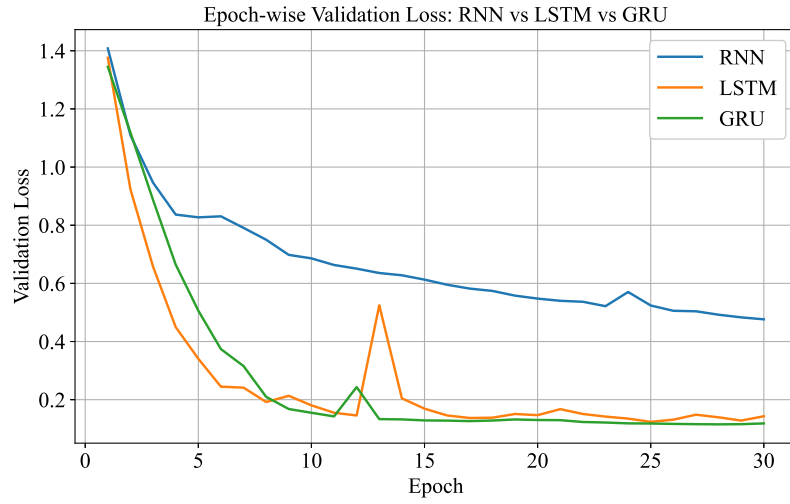


Figure 11: Epoch-wise validation loss comparison.

Inference: The validation loss comparison shows lstm and gru perform way better than rnn wiht a noticably lower loss , because the gated models hace effective retaining of temporal information

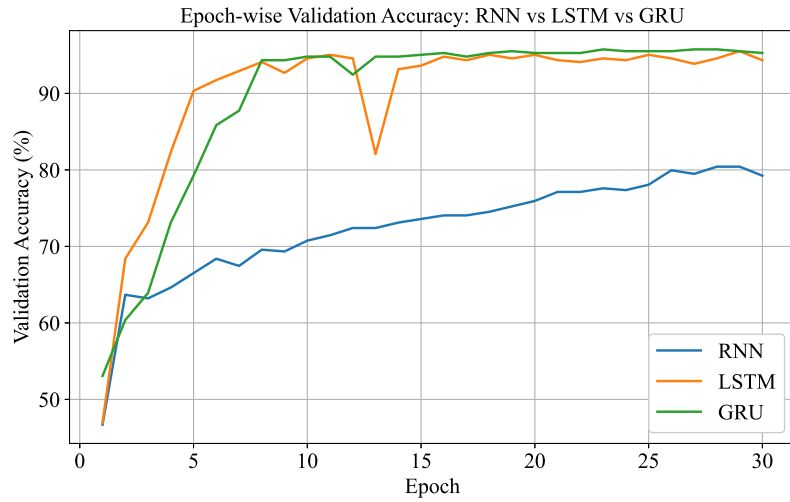


Figure 12: Epoch-wise validation accuracy comparison.

Inference:The validation accuracy comparison shows lstm and gru perform way better than rnn with a noticably higher accuracy , because the gated models hace effective retaining of temporal information

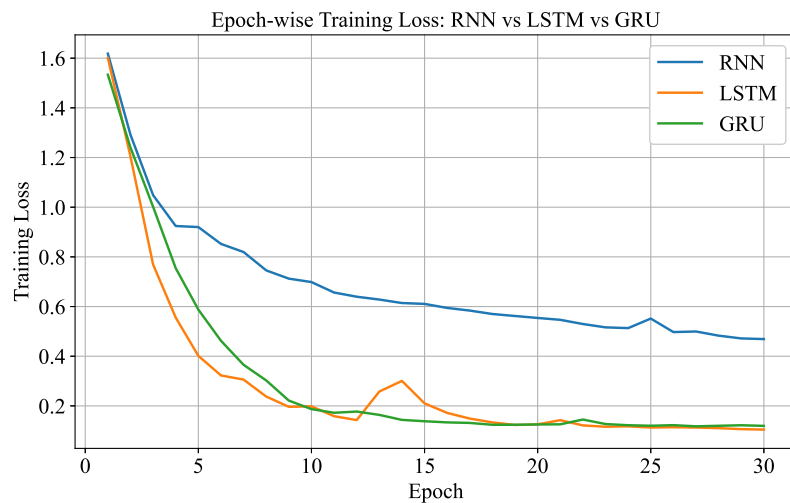


Figure 13: Epoch-wise training loss comparison.

Inference: Similar to the validation loss the training loss shows that the lstm and gru reduce their losses faster than the vanilla rnn .

Required inference: trade-off among;

Predictive Performance ↔ Model Complexity ↔ Training Cost.

Inference: RNN has the fewest parameter count of 1974 whereas the LSTM has the most parameter od 6006 and the GRU having 4758 parameters , but the GRU and the LSTM perform significantly better with higher F1 scores and accuracies . Regardless of the lower parameters the vanilla rnn took the longest to train

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	69.89%	86.72%	83.99%
64	67.76%	89.94%	90.40%
128	66.53%	88.76%	88.51%

Plot 6: Sequence Length vs. Test F1-score

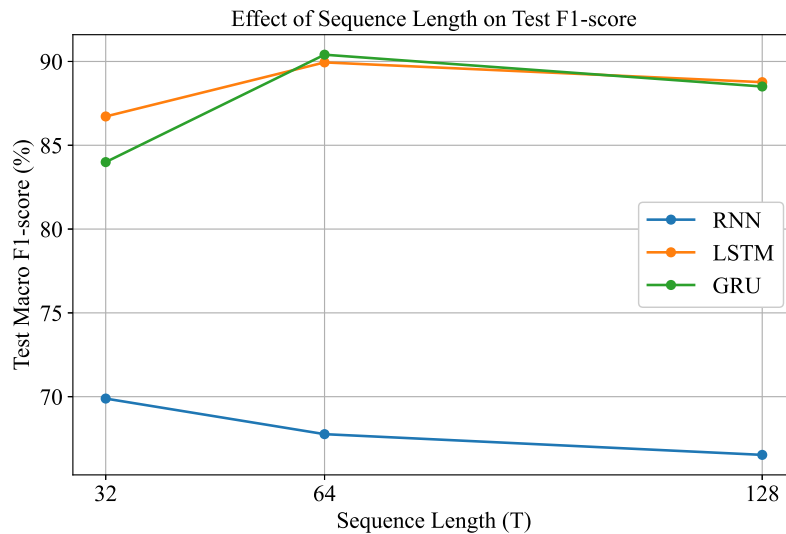


Figure 14: Sequence length versus test F1-score.

Inference: As the sequence lenght progresses it does not seem to help the RNN as its F1 score decreased but the lstm and the gru performed the best at length=64 suggesting a moderate sequence length gave the best performance

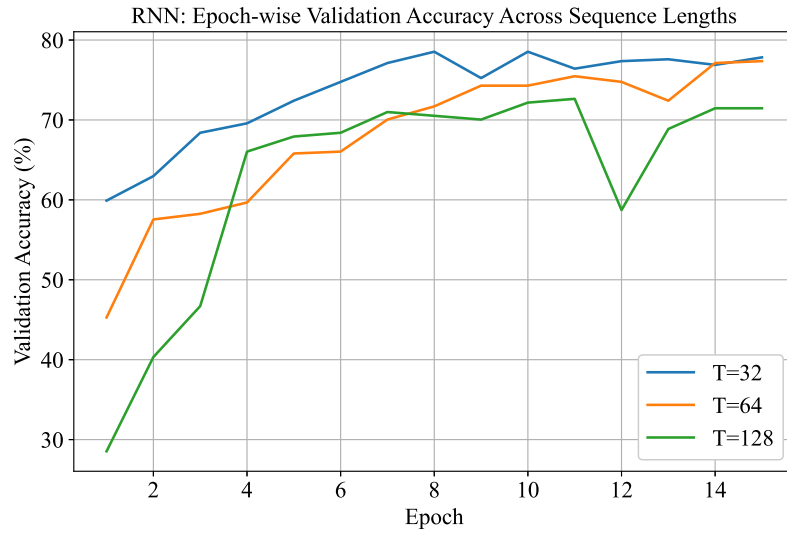


Figure 15: RNN epoch-wise performance by sequence length.

Inference: RNN validation accuracy raises steadily for $T=32$ but shows little fluctuation with $T=64$ and significant fluctuations at $T=128$ observing the $T=128$ performing worse than the lower versions

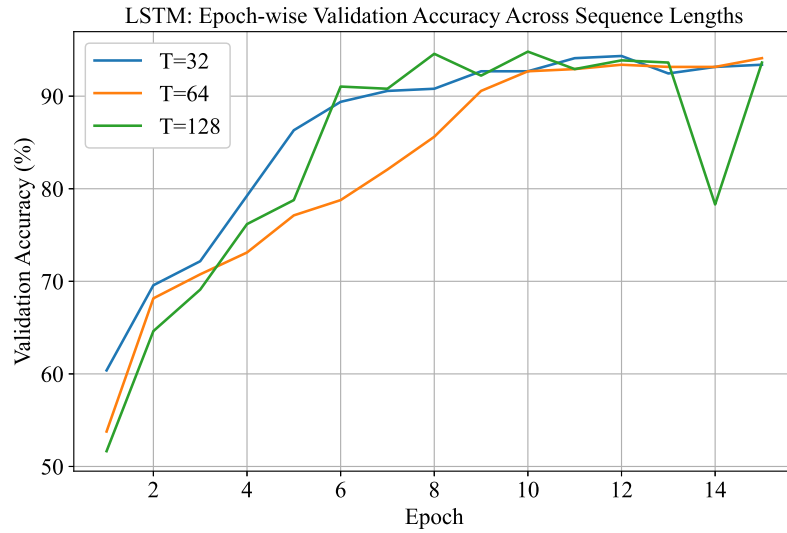


Figure 16: LSTM epoch-wise performance by sequence length.

Inference: The LSTM curves show all the three sequence lengths yielding similar scores while the higher lengths showing some fluctuations across the epochs

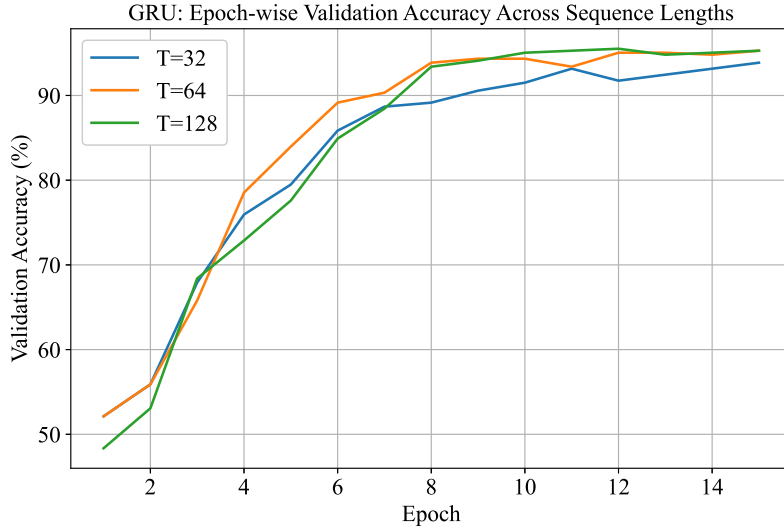


Figure 17: GRU epoch-wise performance by sequence length.

Inference: The GRU curves show the longer sequence lengths yielding higher scores while still showing some fluctuations across the epochs for the higher frequencies

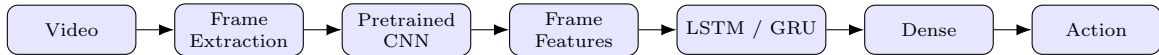
18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Basketball
Actual class : Basketball
Confidence : 0.xx

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



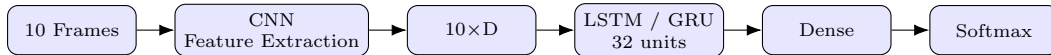
Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

21. CNN-LSTM / CNN-GRU Model

Use the following architecture:



Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

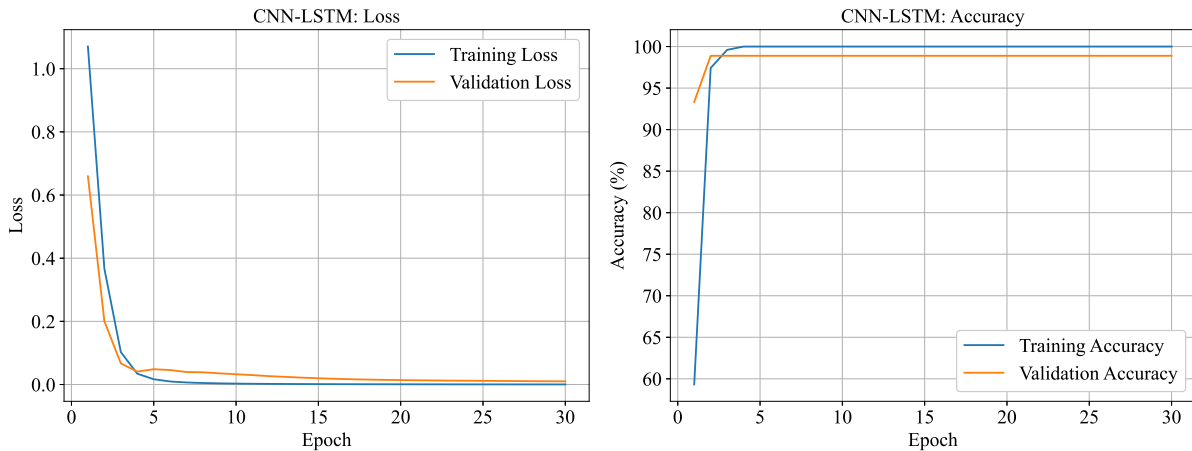


Figure 18: CNN-LSTM video training and validation curves.

Inference: Both the curves show a significant steep decrease in loss and an increase in accuracy raising upto 100 in accuracy and a 0 in loss . The curves are also close suggesting that there is a strong performance but no overfitting .

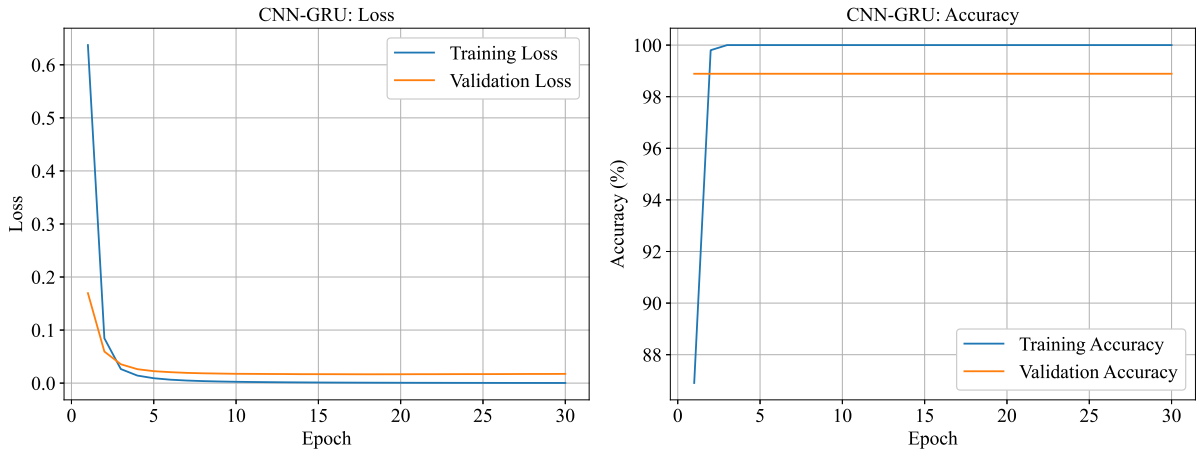


Figure 19: CNN-GRU video training and validation curves.

Inference: Similarly the CNN-GRU also show the rapid convergence achieving 100 accuracy and 0 loss within the first few epochs . The close curves suggesting high generalisation on the selected video test task

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

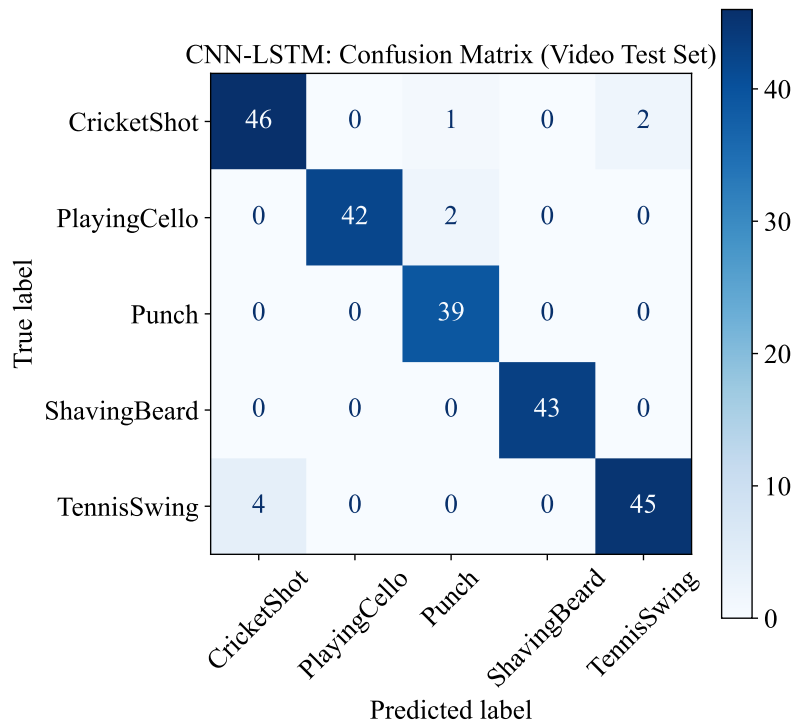


Figure 20: CNN-LSTM video confusion matrix.

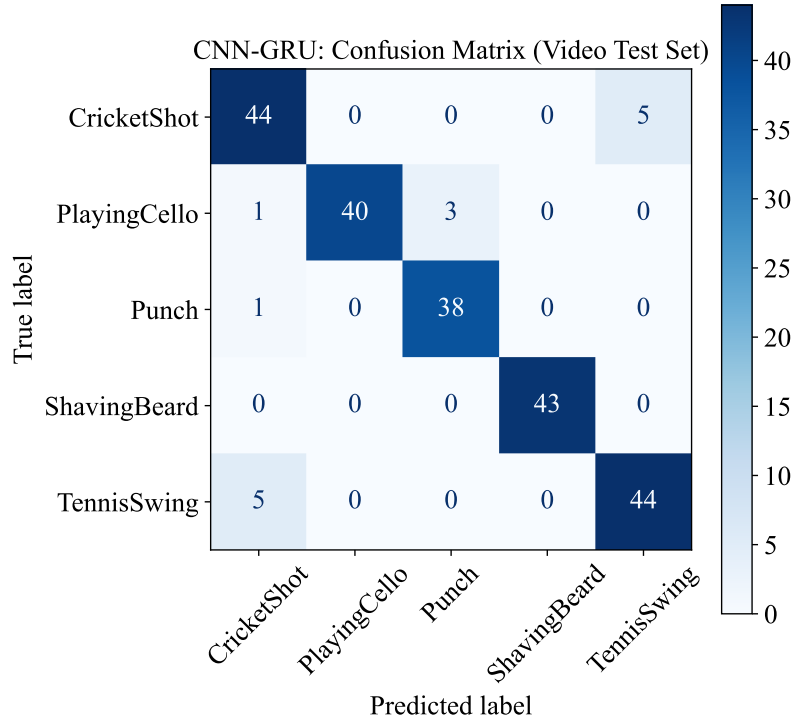


Figure 21: CNN-GRU video confusion matrix.

Inference:.. Both the confusion matrices show very high true values with string diagonal values because of the cnn extracting the spatial information and the lstm or a Gru capturing the temoporal information across the frames

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

Sequence-to-Sequence Training Plot

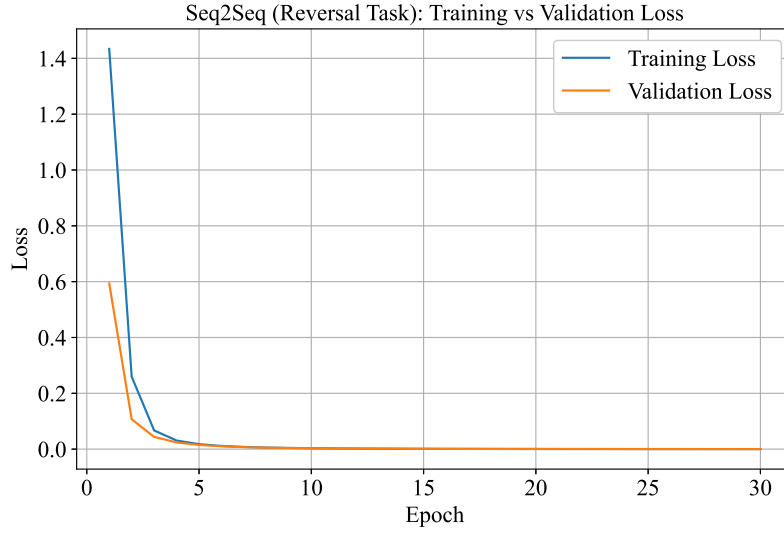


Figure 22: Sequence-to-sequence training loss curve.

Inference: The seq to seq training converges sharply at the first few epochs and achieve near 0 because of the deterministic nature of the reversal task , hence the encoder being able to decipher the mapping quickly

23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Sample	Input Sequence	Predicted Output
1	[4, 4, 0, 6]	[6, 0, 4, 4]
2	[3, 1, 1, 3]	[3, 1, 1, 3]
3	[5, 7, 4, 3]	[3, 4, 7, 5]
4	[1, 5, 5, 4]	[4, 5, 5, 1]
5	[2, 0, 7, 8]	[8, 7, 0, 2]

24. Sequence-to-Sequence Evaluation

- Token accuracy: 100.00%
- Sequence accuracy: 100.00%
- Training loss: 0
- Validation loss: 0
- Test loss : 0.0001

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

Inference: Token accuracy is measured by positions of individual elements whereas sequence accuracy is measured by positions of all the elements in the sequence . Hence one incorrect token weighs more on the sequence accuracy more than the token accuracy .

25. Consolidated Results

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	73.057%	73.107%	72.108%	72.316%	1974
LSTM	89.277%	89.275%	89.375%	89.208%	6006
GRU	90.431%	90.431%	90.550%	90.414%	4758
CNN-LSTM/GRU	95.982%	96.120%	96.234%	96.130%	168677

A separate table must be provided for the sequence-to-sequence task.

26. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data → Preprocessing → RNN/LSTM/GRU → Evaluation

and the video-understanding pipeline:

Video → Frames → CNN Features → LSTM/GRU → Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence → Encoder → Context → Decoder → Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

27. SOURCE CODE

<https://github.com/sriwhatsnext/dllab/tree/main/Week06>

28. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” *EMNLP*, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” *ESANN*, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>